

第2章 分治策略

2.1 分治策略的基本思想

- 实例——二分检索、二分归并排序、Hanio塔
- 分治策略的一般性描述

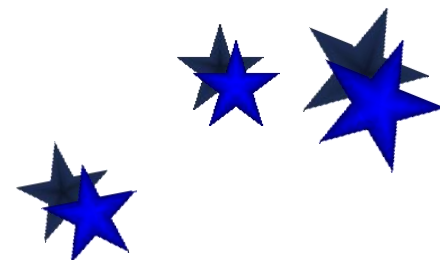
2.2 分治算法的分析技术

- 分治策略的时间复杂度分析方法
- 实例——芯片测试、幂乘算法、快速排序

2.3 改进分治算法的途径

- 通过代数变换减少子问题的个数
- 利用预处理减少递归内部的计算量

2.4 典型实例



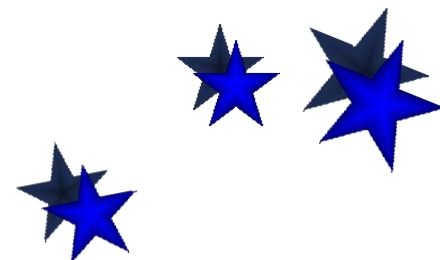
2.1 分治策略的基本思想

■ 分治策略：(Divide and Conquer)

- 将原始问题划分或者归结为 k 个规模较小的子问题
- 递归或迭代求解每个子问题——独立求解
- 将子问题的解综合得到原问题的解

◆ 注意

- 子问题与原始问题性质完全一样
- 子问题之间可彼此独立地求解
- 递归停止时子问题可直接求解



2.1 分治策略的基本思想

分治法解决的问题一般具有以下四个特征：

- (1) 该问题的规模缩小到一定的程度就可以容易地解决。
- (2) 该问题可以分解为若干个规模较小的相同问题。
- (3) 利用该问题分解出的子问题的解可以合并为该问题的解
- (4) 该问题所分解出的各个子问题是相互独立的，即子问题之间不包含公共的子问题。



2.1 分治策略的基本思想

分治法特征说明

特征1：该问题的规模缩小到一定的程度就可以容易地解决。

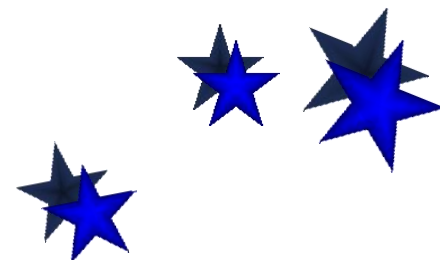
说明：该特征是绝大多数问题都可以满足的，因为问题的计算复杂性都是随着问题规模 n 的增加而增加的，当问题缩小到较小规模则很容易解决——例如1个数的排序、1个或2个数找最大值。



2.1 分治策略的基本思想

特征2：该问题可以分解为若干个规模较小的相同问题。

说明：该特征是使用分治法的前提，它也是大多数问题可以满足的，此特征反映了递归思想的应用。例如：分治法求最大、最小的实现。



2.1 分治策略的基本思想

特征3：利用该问题分解出的子问题的解可以合并为该问题的解。

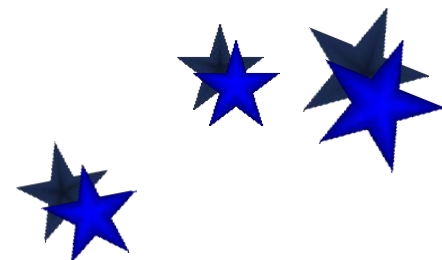
说明：特征3是关键。能否使用分治法完全取决于该问题是否通过子问题解的合并构成整个问题的解。如果某问题具备了特征1、2，而不具备特征3，则可以考虑使用贪心算法或动态规划法。



2.1 分治策略的基本思想

特征4：该问题所分解出的各个子问题是相互独立的，即子问题之间不包含公共的子问题。

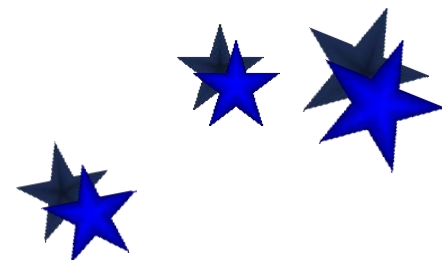
说明：特征4涉及分治的效率，如果各子问题是不独立的则分治要做许多不必要的工作，重复的解公共的子问题，此时虽然可用分治法，但一般使用动态规划较好。



2.1 分治策略的基本思想

- 根据分治法的分割原则，原问题应该分为多少个子问题才较适宜？各个子问题的规模应该怎样才为适当？
 - (1) 这些问题很难给出肯定的回答。
 - (2) 但人们从大量实践中发现，在用分治法设计算法时，最好使子问题的规模大致相同。换句话说，将一个问题分成大小相等的 k 个子问题的处理方法是行之有效的。

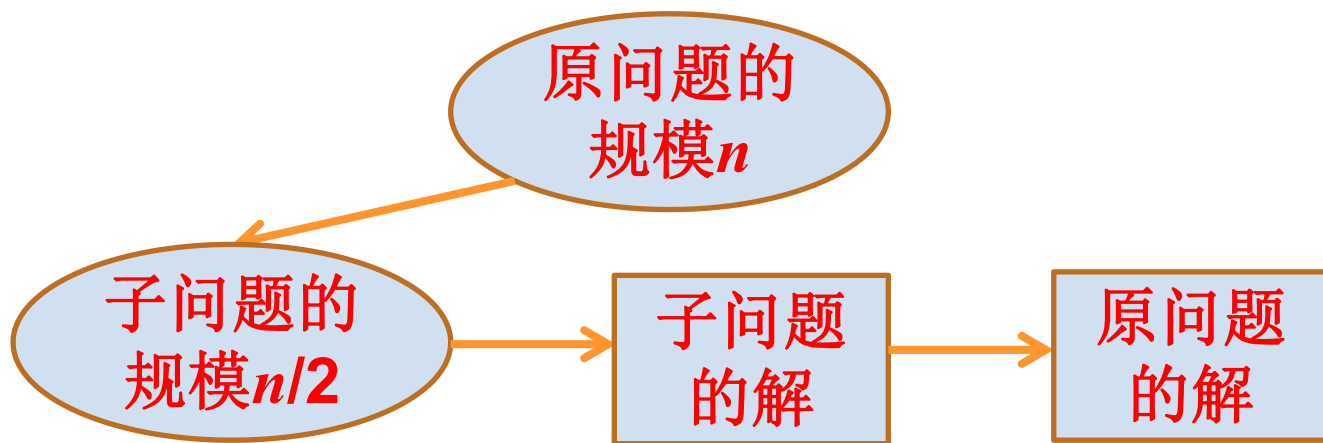
当 $k=1$ 时称为减治法（例如：折半查找）。



2.1 分治策略的基本思想

- 减治法 —例如：折半查找

- (1) 原问题的解只存在于其中一个较小规模的子问题中
- (2) 原问题的解与其中一个较小规模的解之间存在某种对应关系。



- 分治法：对分解的子问题分别求解，需要对子问题的解进行合并。



2.1 分治策略的基本思想

■ 分治策略举例

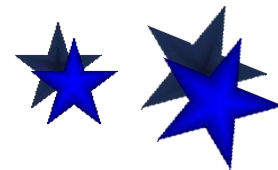
- 例题1：二分检索法——迭代实现

算法 **Binary Search** (T, l, r, x)

输入：数组 T ，下标从 l 到 r ；数 x

输出： j // 若 x 在 T 中， j 为下标；否则为 0

1. $l \leftarrow 1; r \leftarrow n$
2. while $l \leq r$ do
3. $m \leftarrow \lfloor (l+r)/2 \rfloor$ // m 为中间位置
4. if $T[m] = x$ then return m // x 是中位数
5. else if $T[m] > x$ then $r \leftarrow m-1$
6. else $l \leftarrow m+1$
7. return 0

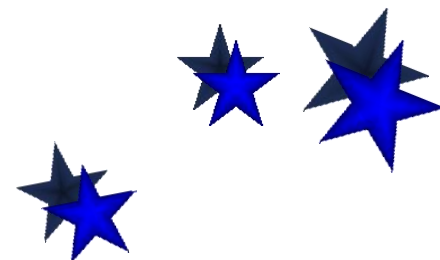


2.1 分治策略的基本思想

- 二分检索法算法设计思想
 - (1) 通过 x 与中间数的比较, 将原问题归结为规模减半的子问题, 如果 x 小于中间数, 则子问题由小于 x 的数构成, 否则子问题由大于 x 的数构成。
 - (2) 当子问题规模为 1 时, 直接比较 x 与 $T[m]$, 若相等则返回 m , 否则返回 0。



是否能够递归实现?



2.1 分治策略的基本思想

- 二分检索法的递归实现

算法 Binary Search (T, l, r, x)

输入：数组 T ，下标从 1 到 r ；数 x

输出： j // 若 x 在 T 中, j 为下标; 否则为 0

1. $l \leftarrow 1; r \leftarrow n$
2. while $l \leq r$ do
3. $m \leftarrow \lfloor (l+r)/2 \rfloor$
4. if $T[m] = x$ then return m // x 恰好等于中位元素
5. else if $T[m] > x$ then Binary Search($T, l, m-1, x$);
6. else Binary Search($T, m+1, r, x$);
7. return 0



2.1 分治策略的基本思想

- 二分检索法的时间复杂度分析

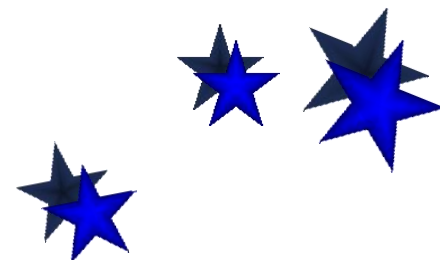
二分检索问题最坏情况下时间复杂度

$$W(n) = W(\lfloor n/2 \rfloor) + 1$$

$$W(1) = 1$$

可以解出

$$W(n) = \lfloor \log n \rfloor + 1$$



2.1 分治策略的基本思想

- 例题2：二分归并排序

算法 **Merge Sort** (A, p, r)

输入：数组 $A[p .. r]$

输出：元素按从小到大排序的数组 A

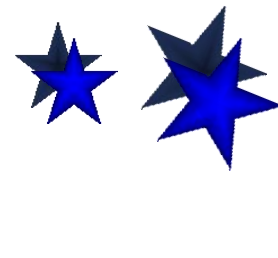
1. if $p < r$

2. then $q \leftarrow \lfloor (p + r)/2 \rfloor$ 对半划分

3. **Merge Sort** (A, p, q) 子问题1

4. **Merge Sort** ($A, q+1, r$) 子问题 2

5. **Merge** (A, p, q, r) 综合解



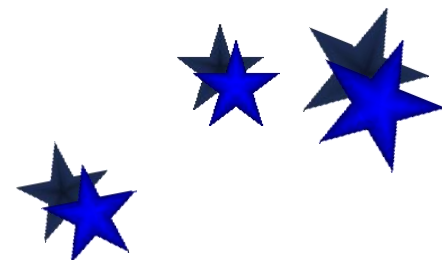
2.1 分治策略的基本思想

- 二分归并排序设计思想

(1) 划分将原问题归结为规模为 $n/2$ 的2个子问题。

(2) 继续划分，将原问题归结为规模为 $n/4$ 的4个子问题。
继续...，当子问题规模为1时，划分结束。

(3) 从规模1到 $n/2$ ，陆续归并被排好序的两个子数组。
每归并一次，数组规模扩大一倍，直到原始数组。



2.1 分治策略的基本思想

- 二分归并排序时间复杂度分析

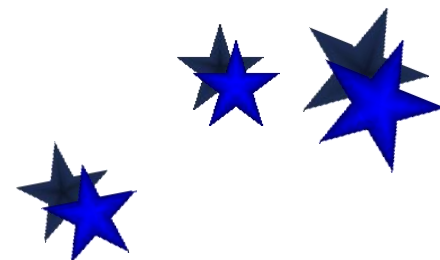
假设 $n=2^k$ 的幂，二分归并排序最坏情况下时间复杂度

$$W(n) = 2W(n/2) + n - 1$$

$$W(1) = 0$$

可以解出

$$W(n) = n \log n - n + 1$$

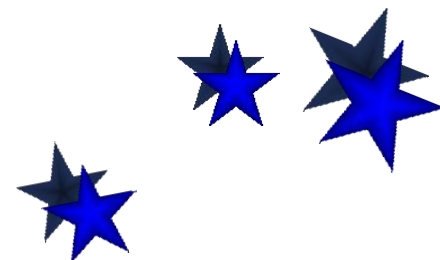


2.1 分治策略的基本思想

- 例题3: Hanoi塔

算法 **Hanoi** (A, C, n) // n 个盘子 A 到 C

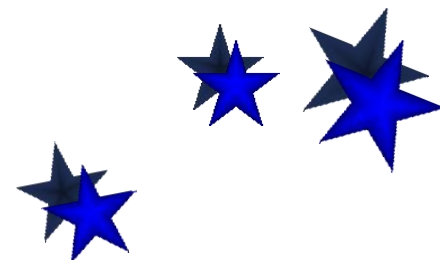
1. if $n=1$ then move (A, C) //1个盘子 A 到 C
2. else **Hanoi** ($A, B, n-1$)
3. move (A, C)
4. **Hanoi** ($B, C, n-1$)



2.1 分治策略的基本思想

- Hanoi塔问题设计思想

- (1) 将原问题归结为规模为 $n-1$ 的2个子问题。
- (2) 继续归约，将原问题归结为规模为 $n-2$ 的4个子问题
- (3) 继续...，当子问题规模为1 时，归约过程截止。
- (4) 从规模 1到 $n-1$ ，陆续组合两个子问题的解。直到规模为 n .



2.1 分治策略的基本思想

- Hanoi塔问题时间复杂度分析

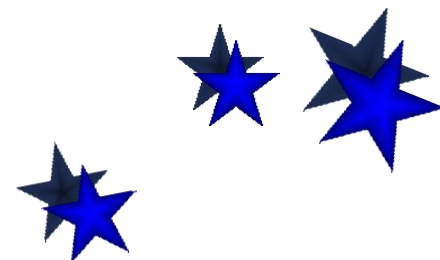
设 n 个盘子的移动次数为 $T(n)$

$$T(n) = 2 T(n-1) + 1,$$

$$T(1) = 1,$$

可以解出

$$T(n) = 2^n - 1$$

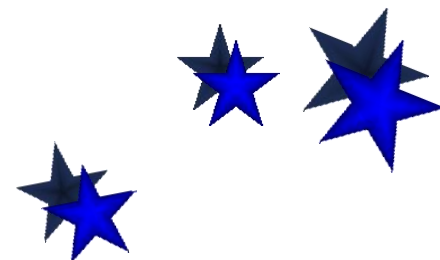


2.1 分治策略的基本思想

- 小结

通过几个例子展示分治算法的特点：

- (1) 将原问题归约为规模小的子问题，**子问题与原问题具有相同的性质。**
- (2) 算法的分析方法：递推方程。
- (3) 子问题规模足够小的时候可直接求解。



2.1 分治策略的基本思想

■ 分治策略算法框架描述

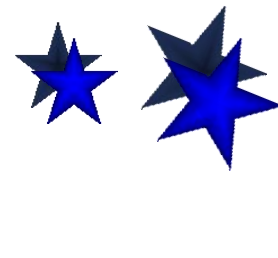
分治算法 **Divide-and-Conquer(P)**

1. if $|P| \leq c$ then $S(P)$.
2. divide P into $P_1, P_2, \dots, P_k$
3. for $i \leftarrow 1$ to k
4. $y_i \leftarrow \text{Divide-and-Conquer}(P_i)$
5. Return Merge($y_1, y_2, \dots, y_k$)

划分

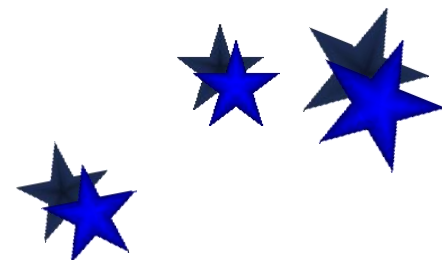
求解子
问题

综合
解



2.1 分治策略的基本思想

- 分治算法设计要点
 - 原问题可以划分或者归约为规模较小的子问题
 - * 子问题与原问题具有相同的性质
 - * 子问题的求解彼此独立
 - * 划分时子问题的规模尽可能均衡
 - 子问题规模足够小时可直接求解
 - 子问题的解综合得到原问题的解
 - 算法实现：递归或迭代



2.2 分治算法的分析技术

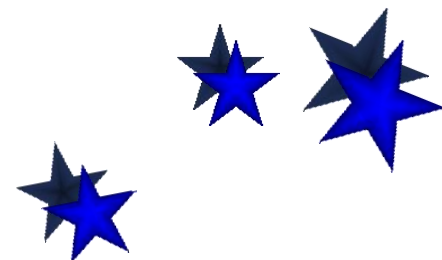
- 分治算法时间分析

时间复杂度函数的递推方程

$$W(n) = W(|P_1|) + W(|P_2|) + \dots + W(|P_k|) + f(n)$$

$$W(c) = C$$

- $P_1, P_2, \dots, P_k$ 为划分后产生的子问题
- $f(n)$ 为划分子问题以及将子问题的解综合得到原问题解的总工作量
- 规模为 c 的最小子问题的的工作量为 C



2.2 分治算法的分析技术

- 两类常见的递推方程

$$f(n) = \sum_{i=1}^k a_i f(n-i) + g(n) \quad (1)$$

$$f(n) = af\left(\frac{n}{b}\right) + d(n) \quad (2)$$

例子:

Hanoi塔, $W(n) = 2W(n-1) + 1$

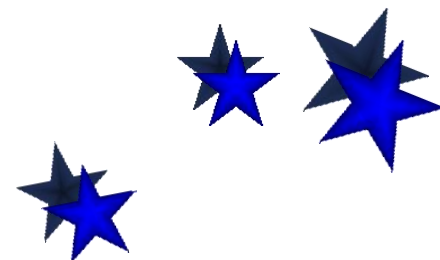
二分检索, $W(n) = W(n/2) + 1$

归并排序, $W(n) = 2W(n/2) + n - 1$

求解方法

方程1: 迭代法、递归树

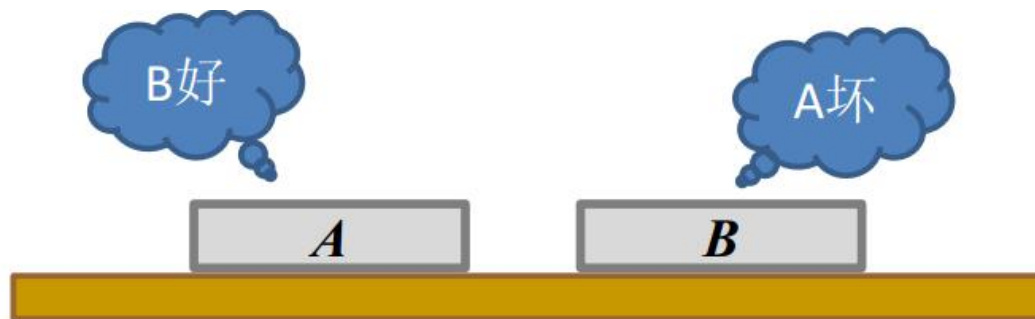
方程2: 迭代法、换元法、递归树、
主定理



2.2 分治算法的分析技术

■ 举例1：芯片测试问题

- 测试方法：将**2**片芯片（**A**和**B**）置于测试台上，互相进行测试，测试报告是“好”或“坏”，只取其一。



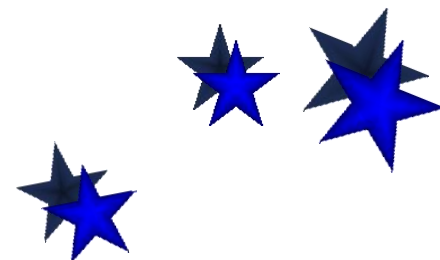
- 假设：好芯片的报告一定是正确的，坏芯片的报告是不确定的（可能会出错）



2.2 分治算法的分析技术

- 测试结果分析

<i>A</i> 报告	<i>B</i> 报告	结论
<i>B</i> 是好的	<i>A</i> 是好的	<i>A, B</i> 都好或 <i>A, B</i> 都坏
<i>B</i> 是好的	<i>A</i> 是坏的	至少一片是坏的
<i>B</i> 是坏的	<i>A</i> 是好的	至少一片是坏的
<i>B</i> 是坏的	<i>A</i> 是坏的	至少一片是坏的



2.2 分治算法的分析技术

- 问题描述

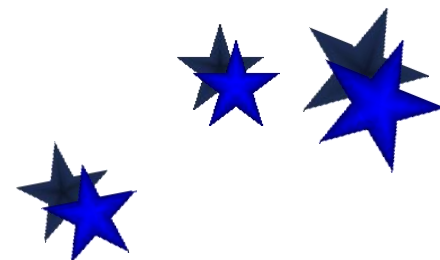
输入：

n 片芯片，其中好芯片至少比坏芯片多 1 片。

问题：

设计前面介绍的测试方法，从 n 片芯片中挑出 1 片好芯片。

要求： 使用最少的测试次数——效率。



2.2 分治算法的分析技术

- 判定芯片 A 的好坏

问题：给定芯片 A ，判定 A 的好坏

方法：用其他 $n-1$ 片芯片对 A 测试。

$n=7$ ：好芯片数 ≥ 4 。

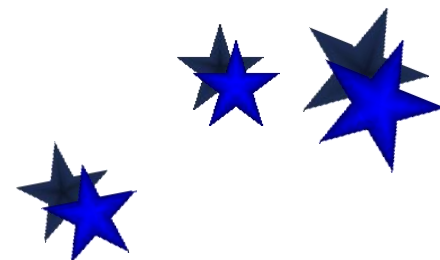
A 好，6 个报告中至少 3 个报“好”

A 坏，6 个报告中至少 4 个报“坏”

n 是奇数：好芯片数 $\geq (n+1)/2$ 。

A 好，至少有 $(n-1)/2$ 个报“好”

A 坏，至少有 $(n+1)/2$ 个报告“坏”



2.2 分治算法的分析技术

$n=8$: 好芯片数 ≥ 5 .

A 好, 7 个报告中至少 4 个报“好”

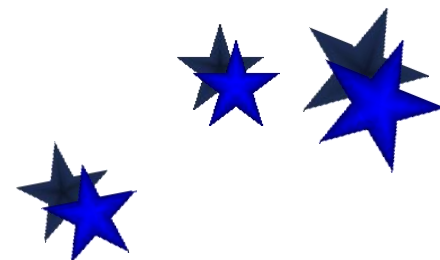
A 坏, 7 个报告中至少 5 个报“坏”

n 是偶数 : 好芯片数 $\geq n/2+1$.

A 好, 至少有 $n/2$ 个报告“好”

A 坏, 至少有 $n/2+1$ 个报告“坏”

结论 : $n-1$ 份报告中,
至少一半报“好”, 则 A 为好芯片
超过一半报“坏”, 则 A 为坏芯片



2.2 分治算法的分析技术

□ 解决策略1：蛮力策略

- 测试方法：任取 1 片测试，如果是好芯片，测试结束；如果是坏芯片，抛弃，再从剩下芯片中任取 1 片测试，直到得到 1 片好芯片。

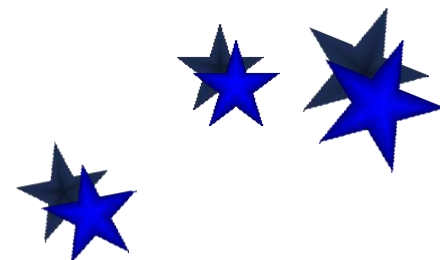
- 时间估计：

第1 片坏芯片，最多测试 $n-2$ 次，

第2 片坏芯片，最多测试 $n-3$ 次，

.....

总计 $O(n^2)$



2.2 分治算法的分析技术

□ 解决策略2：分治策略

- 基本思想：假设 n 为偶数，将 n 片芯片两两一组做测试淘汰，剩下芯片构成子问题，进入下一轮分组淘汰。
- 淘汰规则：
 - "好，好" → 任留1片，进入下轮
 - 其他情况 → 全部抛弃
- 递归截止条件： $n \leq 3$
 - 3片芯片，1次测试可得到好芯片——因为好芯片多于坏芯片。
 - 1 或2片芯片，不再需要测试。



2.2 分治算法的分析技术

- 分治算法的正确性

(1) 命题1: 当 n 是偶数时, 在上述淘汰规则下, 经过一轮淘汰, 剩下的好芯片比坏芯片至少多1片。

证明: 设 A, B 都好的芯片 i 组, A 与 B 一好一坏 j 组, A 与 B 都坏的 k 组. 淘汰后好芯片至少 i 片, 坏芯片至多 k 片。

$$2i + 2j + 2k = n$$

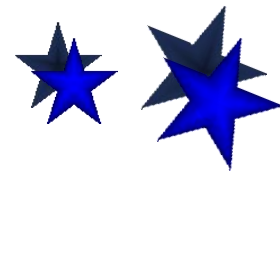
初始芯片总数 n

$$2i + j > 2k + j$$

初始: 好芯片多于坏芯片

$$\longrightarrow i > k$$

子问题与原问题性质相同



2.2 分治算法的分析技术

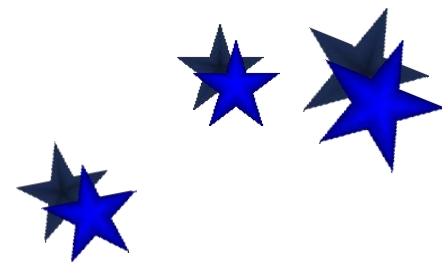
- n 为奇数时的特殊处理：当 n 是奇数时，可能出问题

输入：好 好 好 好 坏 坏 坏

分组：好 好 好 好 坏 坏 坏

淘汰后：好 好 坏 坏

- 处理办法：当 n 为奇数时，增加一轮对轮空芯片的单独测试。
- 结果：如果该芯片为好芯片，算法结束；如果是坏芯片，则淘汰该芯片。



2.2 分治算法的分析技术

• 伪码描述：算法2.3 Test(n)

1. $k \leftarrow n$

2. while $k > 3$ do

分组
淘汰

3. 将芯片分成 $\lfloor k/2 \rfloor$ 组 // 如有轮空芯片，特殊处理

4. for $i=1$ to $\lfloor k/2 \rfloor$ do

if 2片好 then 任取1片留下

else 2片同时丢掉

5. $k \leftarrow$ 剩下的芯片数

6. if $k = 3$

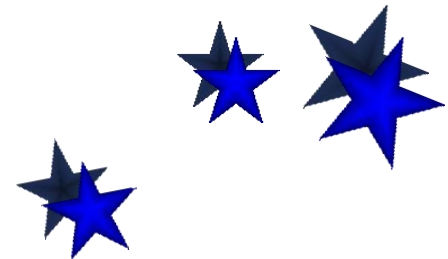
递归
结束

then 任取2片芯片测试

if 1好1坏 then 取没测的芯片

else 任取1片被测芯片

7. if $k = 2$ or 1 then 任取1片



2.2 分治算法的分析技术

- 时间复杂度分析

设输入规模为 n

每轮淘汰后，芯片数至少减半

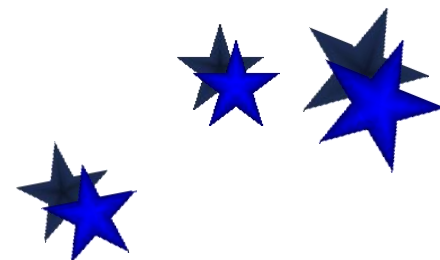
测试次数(含轮空处理): $O(n)$

时间复杂度:

$$W(n) = W(n/2) + O(n)$$

$$W(3) = 1, W(2) = W(1) = 0$$

解得 $W(n) = O(n)$



2.2 分治算法的分析技术

■ 举例2：快速排序

- 基本思想：用首元素 x 作划分标准，将输入数组 A 划分成不超过 x 的元素构成的数组 A_L ，大于 x 的元素构成的数组 A_R 。其中 A_L, A_R 从左到右存放在数组 A 的位置。
- 递归地对子问题 A_L 和 A_R ，进行排序，直到子问题规模为1时停止。
- 划分实例：

27 99 0 8 13 64 86 16 7 10 88 25 90



2.2 分治算法的分析技术

- 一次划分的过程:

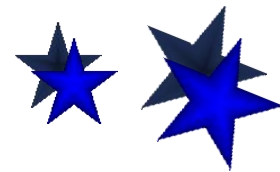
27 **99** 0 8 13 64 86 16 7 10 88 **25** 90
i *j*

27 25 0 8 13 **64** 86 16 7 **10** 88 99 90
i *j*

27 25 0 8 13 10 **86** 16 **7** 64 88 99 90
i *j*

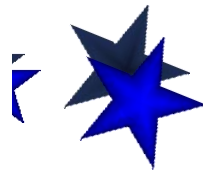
27 25 0 8 13 10 7 **16** **86** 64 88 99 90
j *i*

16 25 0 8 13 10 7 **27** 86 64 88 99 90



2.2 分治算法的分析技术

- 划分算法: **Partition** (A, p, r)
 1. $x \leftarrow A[p]$
 2. $i \leftarrow p$
 3. $j \leftarrow r + 1$
 4. while true do
 5. repeat $j \leftarrow j - 1$
 6. until $A[j] \leq x$ // 不超过首元素的
 7. repeat $i \leftarrow i + 1$
 8. until $A[i] > x$ // 比首元素大的
 9. if $i < j$
 10. then $A[i] \leftrightarrow A[j]$
 11. else return j



2.2 分治算法的分析技术

- 快速排序算法：

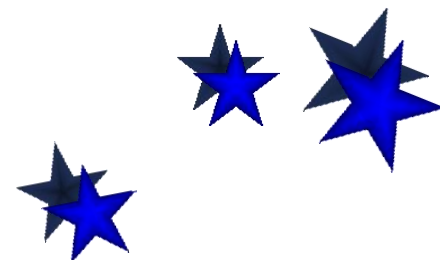
算法 Quicksort (A, p, r)

输入：数组 $A[p..r]$

输出：排好序的数组 A

1. if $p < r$
2. then $q \leftarrow \text{Partition}(A, p, r)$
3. $A[p] \leftrightarrow A[q]$
4. Quicksort ($A, p, q-1$)
5. Quicksort ($A, q+1, r$)

初始置 $p=1, r=n$ ，然后调用上述算法

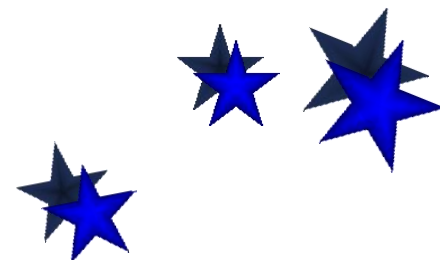


2.2 分治算法的分析技术

- 时间复杂度分析:

最坏情况: $W(n) = W(n-1) + n - 1$
 $W(1) = 0$
 $W(n) = n(n-1)/2$

最好划分: $T(n) = 2 T(n/2) + n - 1$
 $T(1) = 0$
 $T(n) = \Theta(n \log n)$



2.2 分治算法的分析技术

平均时间复杂度分析:

首元素排好序后可能放在 $1, 2, \dots, n$
各种情况概率都为 $1/n$

首元素在位置 1 : $T(0), T(n-1)$

首元素在位置 2 : $T(1), T(n-2)$

....

首元素在位置 $n-1$: $T(n-2), T(1)$

首元素在位置 n : $T(n-1), T(0)$

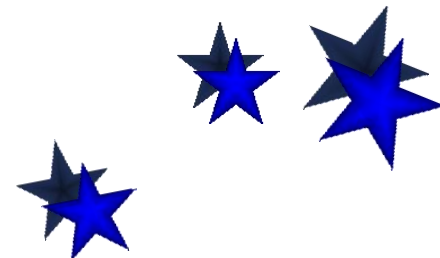
$$T(n) = \frac{2}{n} \sum_{k=1}^{n-1} T(k) + n - 1$$

$$T(1) = 0$$

$$T(n) = \Theta(n \log n)$$

子问题工作量 $2[T(1)+T(2)+\dots+T(n-1)]$

划分工作量 $n-1$



2.2 分治算法的分析技术

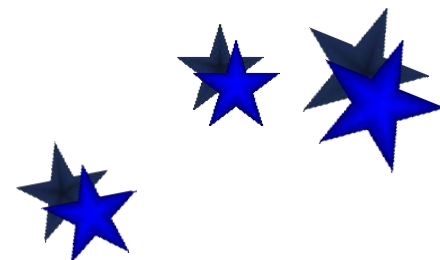
均衡划分的时间复杂度

(1) 均衡划分：每次划分后，子问题1和子问题2的规模比不变。

例如为 1:9

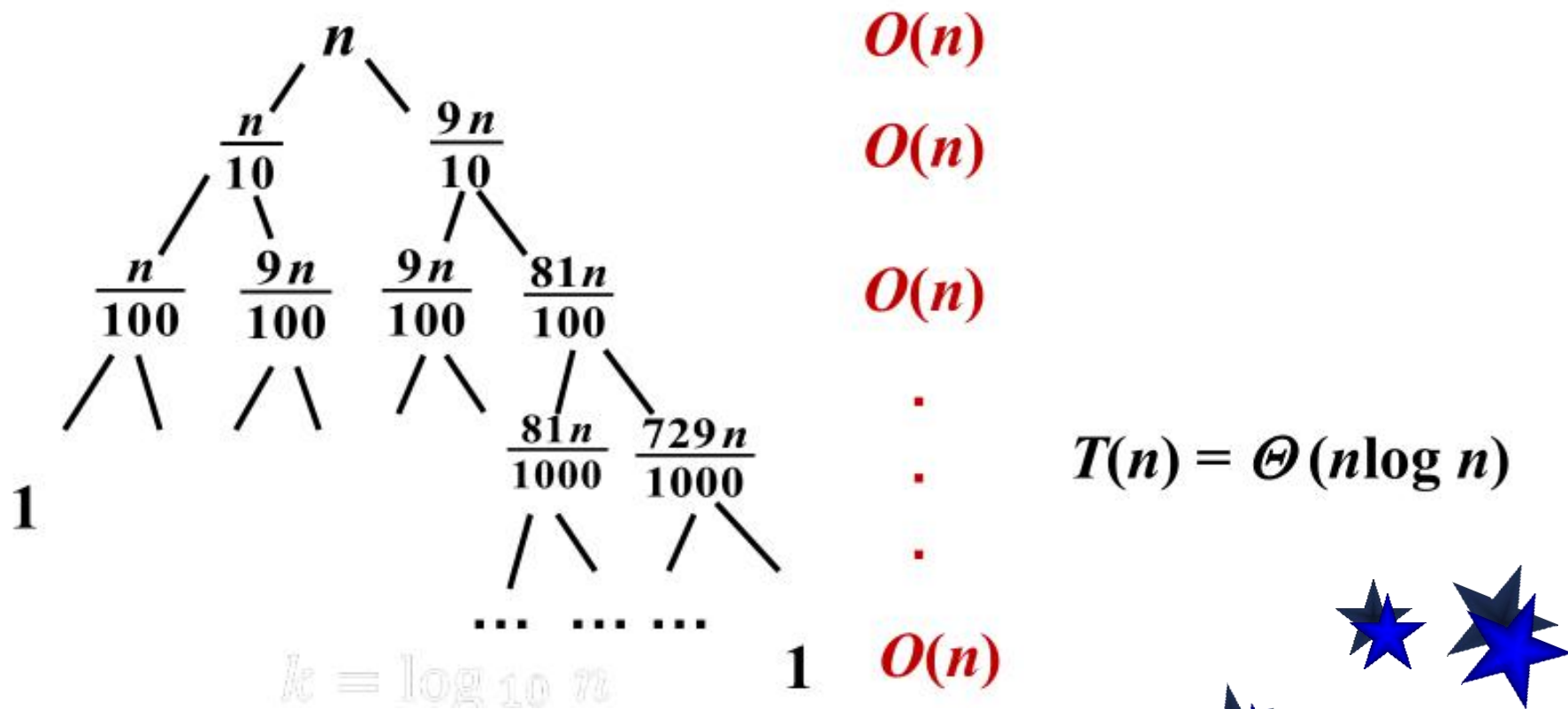
$$T(n) = T(n/10) + T(9n/10) + n$$

$$T(1) = 0$$



2.2 分治算法的分析技术

使用递归树法求解



2.2 分治算法的分析技术

■ 举例3：幂乘算法

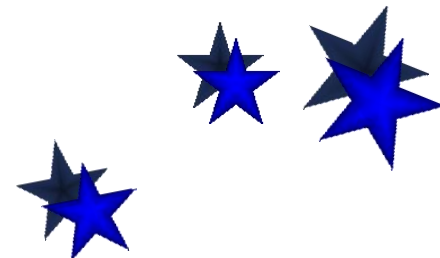
□ 输入： a 为给定实数， n 为自然数

□ 输出： a^n

□ 传统算法：顺序相乘

• $a^n = (...(((a*a)a)a)...)a$

• 乘法次数： $\Theta(n)$



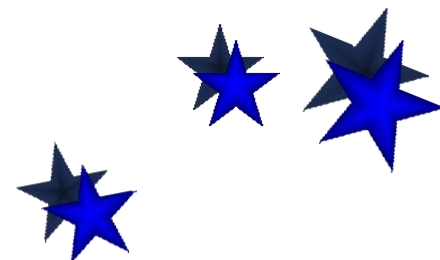
2.2 分治算法的分析技术

□ 分治算法——划分

n 为偶数 $\underbrace{a \dots a}_{n/2 \text{个}} \mid \underbrace{a \dots a}_{n/2 \text{个}}$

n 为奇数 $\underbrace{a \dots a}_{(n-1)/2 \text{个}} \mid \underbrace{a \dots a}_{(n-1)/2 \text{个}} \mid a$

$$a^n = \begin{cases} a^{n/2} \times a^{n/2} & n \text{ 为偶数} \\ a^{(n-1)/2} \times a^{(n-1)/2} \times a & n \text{ 为奇数} \end{cases}$$



2.2 分治算法的分析技术

- 分治策略算法描述

算法: **MiCheng**(a, n)

输入: a, n

输出: a^n

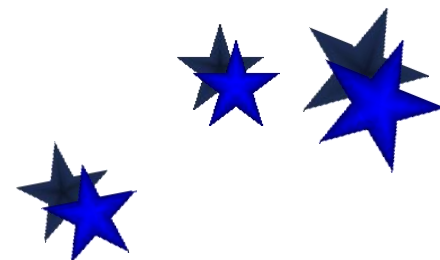
1. if $n=1$ then $y \leftarrow a$
2. else
3. $y \leftarrow \text{MiCheng}(a, n/2);$
4. $y \leftarrow y * y$
5. if(n 是奇数) then $y = y * a$
6. return(y)

以乘法作为基本运算

- 子问题规模: 不超过 $n/2$
- 两个规模近似 $n/2$ 的子问题完全一样, 只要计算1次

$$W(n) = W(n/2) + \Theta(1)$$

$$W(n) = \Theta(\log n)$$



2.2 分治算法的分析技术

□ 幂乘算法的应用——求Fibonacci 数列的第 n 项

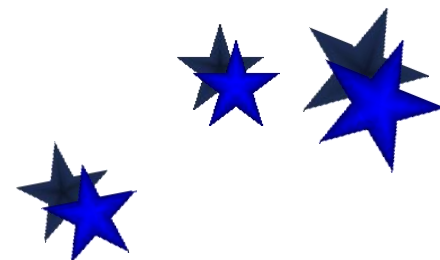
- Fibonacci 数列: 1, 1, 2, 3, 5, 8, 13, 21, ...
满足 $F_n = F_{n-1} + F_{n-2}$

增加 $F_0=0$, 得到数列: 0, 1, 1, 2, 3, 5, 8, 13, 21, ...

问题: 已知 $F_0=0, F_1=1$, 给定 n , 计算 F_n

通常算法: 从 $F_0, F_1, \dots$ 开始, 根据递推公式
 $F_n = F_{n-1} + F_{n-2}$, 陆续相加可得 F_n

时间为: $\Theta(n)$



2.2 分治算法的分析技术

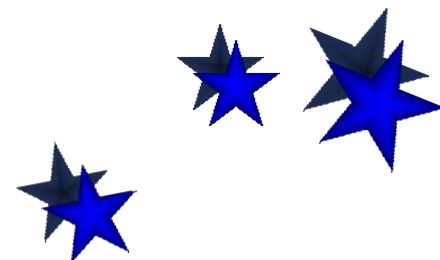
□ Fibonacci 数列的性质

- 定理1: 设 $\{F_n\}$ 为 Fibonacci 数构成的数列, 那么

$$\begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n$$

归纳法证明

$$n=1, \text{ 左边} = \begin{bmatrix} F_2 & F_1 \\ F_1 & F_0 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} = \text{右边}$$



2.2 分治算法的分析技术

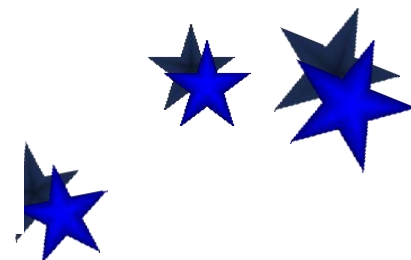
假设对任意正整数 n ，命题成立，即

$$\begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n$$

那么 $\begin{bmatrix} F_{n+2} & F_{n+1} \\ F_{n+1} & F_n \end{bmatrix} = \begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$

$$= \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{n+1}$$

归纳假设代入



2.2 分治算法的分析技术

- 利用矩阵的幂乘计算Fibonacci 数列的第 n 项

令矩阵 $M = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$, 用乘幂算法计算 M^n

时间复杂度:

- 矩阵乘法次数 $T(n) = \Theta(\log n)$
- 每次矩阵乘法需要做 8 次元素相乘
- 总计元素相乘次数为 $\Theta(\log n)$

